

AI 기반 코드 분석

☉ Category	Security Analysis
≡ Description	정적 분석으로 잡지 못하는 취약점을 Source to Sink 경로 추적으로 파악
123 ID	Func-03
⚙ Status	시작 전

Overview

- 목표 : 단순한 패턴 매칭으로는 찾기 어려운 복잡한 보안 위협을 탐지, AI를 활용해 사용자 입력값이 위험한 함수에 도달하기까지의 경로를 추적하여 실질적인 취약점 존재 여부 증명
- 대상 사용자 : 소스 코드 내의 논리적인 데이터 흐름 결함을 파악하고 싶은 보안 전문가 및 개발자

User Flow & Logic

1. 정적 분석 단계에서 생성된 코드 구조 데이터를 바탕으로 데이터의 흐름이 시작되는 지점들을 파악
2. 외부 입력값이 변수를 거쳐 시스템의 핵심 기능에 도달하는 모든 경로를 AI가 추적
3. AI가 경로 사이에 적절한 '검증 로직'이나 '필터링'이 있는지 확인하여, 공격자가 조작한 데이터가 위험 지점까지 도달할 수 있는지 판단
4. 정적 분석의 룰셋에는 없지만, 데이터 흐름상 위험하다고 판단되는 새로운 변종 취약점을 찾아냄
5. 취약점이 어떻게 악용될 수 있는지에 대한 논리적인 공격 경로를 사용자에게 설명

Data Specification

Input

- 보안 위험 지점 목록 : 리스트 (데이터가 도달하면 위험한 Sink 함수들의 목록)

Output

- 위험 경로 : 리스트 (Source부터 Sink까지 이어지는 코드 라인들의 경로)
- 취약점 확정 여부 : 예 / 아니오 (실제로 공격 가능한 경로인지에 대한 AI의 판단)
- 분석 근거 로그 : String (왜 이 경로가 위험하다고 판단했는지에 대한 논리적 근거)

Exception Handling

- 순환 참조 오류 : 코드 경로가 무한 루프에 빠져 분석이 끝나지 않을 경우, 최대 탐색 깊이를 제한하고 사용자에게 알림
- 분석 라이브러리 부재 : 특정 언어의 특수 구문을 AI가 해석하지 못할 경우, 해당 구간을 제외하고 분석 가능한 범위까지만 결과 제공
- 과도한 경로 수 : 분석해야 할 경로가 너무 많아 시간이 오래 걸릴 경우, 위험도가 높은 경로를 우선순위로 먼저 분석

Security Requirements

- 분석 격리 : AI가 코드 경로를 분석하는 과정에서 실제 코드를 실행하지 않도록 하여 분석 환경의 안전 유지
- 데이터 최소화 : 분석에 필요한 논리 구조 데이터만 활용, 주석이나 민감한 상수는 제거
→ 학습 모델 유출 방지
- 추론 신뢰성 : AI가 생성한 공격 경로가 허구일 가능성 고려, 실제 코드 라인과 대조할 수 있는 링크 제공